Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Dersin yapısı
  - Dönemin son dersi de öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırma ile ilgili bir makaleyi özetledi; hoca sunum sonunda yorum yaptı. Toplam beş sunum yapıldı. Sunumların hepsi İngilizce yapıldığı ve altyazı çoğu yerde anlamsız tekrara düştüğü için içerik büyük ölçüde slaytlardan kuruldu.
- Sunum 1: Dil modeliyle metin sıkıştırma (“Compression Through Language Modeling”) (ekrandan, 00:07:20, 00:10:00, 00:17:35, 00:18:15, 00:20:00, 00:20:15, 00:21:38)
  - Veri sıkıştırmanın ilkesi veriyi olabildiğince az bitle saklamaktır; çoğu zaman bu, artıklığın (redundancy) atılması demektir. Özgün veri ile açılmış veri aynıysa sıkıştırma kayıpsız, farklıysa kayıplıdır. Metin sıkıştırma kayıpsız olmak zorundadır.
  - Dil modeli (LM), bir kelime dizisinin cümlede geçme olasılığını istatistiksel ve olasılıksal tekniklerle belirler. N-gram, n kelimelik dizidir: unigram (tek kelime), bigram (iki kelime), trigram (üç kelime). N-gram modeliyle bir dizinin son kelimesinin önceki kelimeler verildiğinde olasılığı ya da bütün dizinin olasılığı kestirilir.
  - Belge, derlem (corpus) ve treebank (kelimeleri sözcük türü ve cümle yapısıyla etiketlenmiş derlem) kavramları tanıtıldı; koşullu olasılık $P(A \mid B)$ hatırlatıldı.
  - Huffman kodlaması (David A. Huffman, 1952): bütün sembollerin sayıları toplanır; en az sayılı iki sembol yeni bir bileşik sembolde birleştirilir, dallar 0 ve 1 ile etiketlenir; tek sembol kalana kadar tekrarlanır.
  - Sıkıştırma algoritması 5 adımdır: unigram, bigram ve trigram dil modeli eğit; her bigram için üçüncü kelimenin gözlenmiş bir kelime ya da UNK olma olasılığını bul; her bigram için üçüncü kelimenin dağılımına göre bir Huffman ağacı kur; kelimeler için bir unigram Huffman ağacı kur; kelimeleri Huffman bitleriyle değiştir, bilinmeyen kelimede UNK kodlanıp daha düşük dereceli modele, unigram da bilmiyorsa karakter düzeyinde Huffman sıkıştırmaya geçilir.
  - Uzantılar: belirli bağlamlar (metni spor, siyaset, iş gibi bir konuya sınıflandırıp o konunun sözlüğünü kullanmak) ve kayan bağlam penceresi (pencereye giren kelimenin sayısı artırılır, çıkanınki azaltılır; yerel bağlam daha iyi kullanılır).
  - Sonuç: yöntem gzip ile rekabet edebilir düzeydedir (gzip’ten yaklaşık %1-2 daha büyük çıktı verir), bzip2 her dosyada belirgin biçimde daha iyidir.
- Sunum 2: DeepZip, tekrarlayan ağlarla kayıpsız sıkıştırma (ekrandan, 00:24:15, 00:30:00, 00:37:35, 00:38:45, 00:40:00)
  - Arithmetic kodlama, bilinen bir dağılım için entropi sınırına ulaşan verimli bir yöntemdir. Dağılım bilinmiyorsa uyarlamalı (adaptive) türleri koşullu k-gram dağılımını öğrenerek sıkıştırır. LSTM ve GRU tabanlı modeller uzun vadeli bağımlılıkları yakalamada ve sonraki karakteri tahmin etmede iyidir.
  - DeepZip iki bloktan oluşur: RNN tabanlı olasılık kestiricisi (geçmiş verildiğinde sonraki sembolün koşullu olasılığı) ve arithmetic kodlayıcı. Elde edilen sıkıştırma boyutu, RNN kestiricisinin normalize çapraz entropi kaybına çok yakındır.
  - Çerçeve: RNN alfabe üzerinde bir dağılım üretir ve arithmetic kodlayıcıya verir; arithmetic kodlayıcı, aralık durumunu tutan bir sonlu durum makinesi (FSM) gibi düşünülür; kestirici nedensel (causal) olmalıdır, yani yalnız önceden kodlanmış sembolleri kullanır; ağırlık güncellemesi yapılıyorsa kodlayıcıda ve kod çözücüde birebir aynı yapılmalıdır.
  - Sentetik veri: i.i.d. kaynaklar ve Markov-k kaynakları (entropisi bilinen, sıfır entro-

pili kaynaklar); entropi sıkıştırma oranı için alt sınır olarak kullanılır. Vanilla RNN (DeepZip-ChRNN) 30 adımdan uzak bağımlılıkları öğrenemez. DeepZip-ChGRU ve özellik tabanlı DeepZip-Feat (son 50 sembol ve 4-gram bağlam sayıları) Markov-k kaynaklarında gzip ve uyarlamalı arithmetic kodlayıcıdan iyidir.
  - Gerçek veri: PRNG dizilerinde DeepZip-Feat çok başarılıdır (100 MB → 500 KB); metin verisinde (Hutter Prize, EnWiki-9) en iyi özel sıkıştırıcı CMIX'in gerisinde kalır. Bunun nedeni metinde çok uzun bağımlılıkların bulunmaması olarak açıklandı.
- Sunum 3: Çok katmanlı görüntü temsili ve görüntü sıkıştırmaya uygulanması (ekrandan, 00:42:00, 00:46:05, 00:50:00, 00:53:12)
  - Görüntü katmanların toplamı olarak ayrıştırılır: parçalı düzgün (cartoon, kenar) katmanı ve doku (texture) katmanı; kalan kısım gürültüye benzer. Her katman farklı bir dönüşüm ya da tabanla, farklı bit hızında, bağımsız kodlanır.
  - Ana döngü: $R^n = \hat{R}^n + R^{n+1}$, $\hat{R}^n = \sum_{j \in E_n} q_j^n \psi_j^n$. Her adımda kalan hatayı b_n bitle en iyi yaklaşıklayan taban, bir taban kütüphanesinden seçilir.
  - Örnek: birinci yineleme parçalı düzgün yoğunluk değişimlerini çok az dalgacık (wavelet) katsayısıyla kodlar ($\hat{R}^0$); ikinci yineleme $R_1 = I - \hat{R}^0$ kalan hatasını yerel kosinüs tabanıyla kodlar (doku katmanı); üçüncü yinelemede doku da çıkarılmıştır, $R_2 = R_1 - \hat{R}^1$ rastgele gürültüye benzer.
  - Maliyet fonksiyonu gerçek skaler niceleme ve entropi kodlamanın maliyetini yaklaşık verir ama çok daha hızlı hesaplanır. Deneylerde dalgacık kodlayıcı çatı kiremitleri gibi dokuları koruyamazken çok katmanlı yöntem dokuyu daha iyi korur; palyaço görüntüsünde SPIHT'ten görsel olarak daha iyi sonuç verir.
- Sunum 4: Cep telefonunda Burrows-Wheeler dönüşümü, öne taşıma (move-to-front) ve arithmetic kodlama ile metin sıkıştırma (ekrandan, 01:06:30, 01:10:00, 01:11:15, 01:15:05)
  - Dört şema karşılaştırıldı: I yalnız arithmetic kodlama; II MTF + arithmetic kodlama; III BWT + arithmetic kodlama (MTF yok); IV BWT + MTF + arithmetic kodlama.
  - Arithmetic kodlama girdideki sembol dizisini $[0, 1)$ aralığında tek bir sayıyla değiştirir. MTF kendisi sıkıştırmaz ama artıklığı azaltır: her sembol, yakın zamanda kullanılan semboller listesindeki indisiyle değiştirilir ve sonra listenin başına taşınır. BWT tersinir, blok sıralamalı bir dönüşümdür.
  - Sıkıştırma oranı: $CR = \dfrac{\text{sıkıştırılmış boyut}}{\text{özgün boyut}} \times 100\%$; küçük oran daha iyidir.
  - Sonuç: en iyi sonuç Şema IV'ten alındı: en iyi oran %32,62, en kötü oran %63,11 (ekrandan, 01:15:05).
- Sunum 5: Akıllı sözlük tabanlı kodlama (IDBE) ile metin sıkıştırma (ekrandan, 01:20:00, 01:27:30, 01:30:00, 01:31:55)
  - Fikir: metni klasik bir sıkıştırıcıdan (BWT tabanlı) önce tersinir bir dönüşümle ara bir biçime çevirmek. Önişleme sıkıştırma verimini artırır.
  - Önceki yöntemler: Star encoding (sık kelimeler sözlüğü; her kelime mümkün olduğunca çok harfi * ile değiştirilmiş eşdeğeriyle yazılır; karakter sayısı aynı kalır ama metin yıldızlarla dolar) ve LIPT (Length Index Preserving Transform).
  - Sözlük kurma: kelimeler çıkarılır, sayılır, sıklığa göre azalan sırada dizilir; ilk 218 kelimeye ASCII 33-250 aralığındaki tek karakter, kalanlara bu aralıktan iki, sonra üç, gerekirse dört karakterlik permütasyonlar verilir.
  - Kodlama: kod uzunluğu ASCII 251-254 ile işaretlenir (251 = 1 uzunluk, 252 = 2 uzunluk, ...); boşluk yokluğu ASCII 255 ile belirtilir; metindeki 251-255 karakterleri iki kez yazılarak işaretten ayrılır.
  - Sonuç (Canterbury derlemi): IDBE ile BWT çoğu metin dosyasında karakter başına biti (BPC) düz BWT'ye ve Star encoding'e göre düşürür (örneğin alice29.txt 2,45 → 2,11).

Hocanın Özellikle Vurguladığı Kısımlar

- DeepZip'in kullandığı veri
  - Hoca, giriş slaytında 3B sanal gerçeklik verisi geçtiği için çalışmanın yalnız 3B veride mi yapıldığını sordu. Sunumun sonunda yöntemin farklı veri setlerinde (metin, kromozom gibi biyoenformatik verisi, Wikipedia verisi) denendiğini ve başarısının iyi

olduğunu belirtti.
- Çok katmanlı sunumun konusu
  - Hoca bu makalenin görüntü işleme alanında SPIHT algoritmasıyla ilgili bir çalışma olduğunu belirtti.

Kısa Tekrar Notları

- Dil modeliyle sıkıştırma: her bigram için üçüncü kelimenin Huffman ağacı; bilinmeyende UNK → unigram → karakter Huffman.
- Huffman örneği: A 0,25, B 0,16, C 0,22, D 0,20, E 0,17 → ortalama uzunluk 2,33 bit/sembol, entropi ≈ 2,30 bit/sembol.
- DeepZip: RNN olasılık kestiricisi + arithmetic kodlayıcı; nedensellik ve aynı ağırlık güncellemesi şartı; Markov-k ve PRNG'de çok iyi, metinde CMIX'in gerisinde.
- Çok katmanlı görüntü: dalgacık (düzgün bölgeler) + yerel kosinüs (doku) + gürültü; $R^n = \hat{R}^n + R^{n+1}$.
- BWT + MTF + arithmetic: en iyi şema, en iyi oran %32,62; MTF tek başına sıkıştırmaz.
- IDBE: sıklığa göre sözlük, ASCII 33-250 kodları, 251-254 uzunluk işareti, 255 boşluk yokluğu; BWT'den önce önişleme.

Detaylı Açıklamalar

- Huffman ağacı örneği (ekrandan, 00:17:35)
  - Olasılıklar: A = 0,25, B = 0,16, C = 0,22, D = 0,20, E = 0,17. Birleştirmeler: E + B = 0,33; C + D = 0,42; EB + A = 0,58; 0,58 + 0,42 = 1.
  - Kodlar: A = 01, B = 001, C = 10, D = 11, E = 000.
  - Ortalama uzunluk: $2(0{,}25) + 3(0{,}16) + 2(0{,}22) + 2(0{,}2) + 3(0{,}17) = 2{,}33$ bit/sembol.
  - Entropi: $-\sum p_i \log_2 p_i \approx 2{,}30$ bit/sembol. İki değer python ile doğrulandı (entropi 2,3026).
- Dil modeli sonuç tablosu (ekrandan, 00:21:38; boyutlar byte)
  - Treebank-sent.test.txt: özgün 558324, bzip2 145853, gzip 198522, metin tabanlı 202186.
  - Treebank-sent.validate.txt: 554908, 147401, 199373, 201645.
  - Blip-sent.test.txt: 5931314, 1514353, 2105318, 2151026.
  - Blip-sent.validate.txt: 5957667, 1519737, 2112540, 2134206.
  - Python ile hesaplanan oranlar: bzip2 yaklaşık %26, gzip yaklaşık %36, metin tabanlı yöntem yaklaşık %36; metin tabanlı çıktı gzip'ten %1-2 büyüktür.
- DeepZip arithmetic kodlama şekli (ekrandan, 00:30:00)
  - (0,6; 0,2; 0,1; 0,1) dağılımlı i.i.d. kaynakta (0; 2; 3) dizisi: $[0; 0{,}6) \to [0{,}48; 0{,}54) \to [0{,}534; 0{,}54)$.
- DeepZip tabloları (ekrandan, 00:37:35, 00:38:45; her veri 100 MB)
  - Markov-k (ChGRU / Feat / gzip / Adaptive-AE): Markov-20 200 KB / 200 KB / 5 MB / 2 MB; Markov-30 1 MB / 400 KB / 40 MB / 87 MB; Markov-50 30 MB / 500 KB / 63 MB / 100 MB; Markov-80 100 MB / 2 MB / 69 MB / 100 MB; Markov-100 100 MB / 38 MB / 100 MB / 100 MB; Markov-140 hepsinde 100 MB.
  - Gerçek ve PRNG veri (ChGRU / Feat / gzip / en iyi özel): PRNG-31-13 32 MB / 500 KB / 95 MB / -; PRNG-52-21 85 MB / 1 MB / 100 MB / -; PRNG-73-31 100 MB / 62 MB / 100 MB / -; Hutter-Prize (100 MB) 18 MB / 16,5 MB / 34 MB / 15,9 MB (CMIX); EnWiki-9 (1 GB) 178 MB / 168 MB / 330 MB / 154 MB (CMIX); Chromosome-1 (240 MB) 48 MB / 42 MB / 57 MB / 49 MB (MFCompress).
- BWT ve MTF örneği (ekrandan, 01:06:30, 01:10:00, 01:11:15)
  - Girdi "fathimahas". Adım 1: dize birer karakter sola döndürülür; adım 2: satırlar sıralanır; adım 3: son sütun alınır.
  - Slayttaki çıktı "aaafhhimst"tir. MTF (alfabe a-z başlangıç listesi) bu diziyi 0, 0, 0, 5, 7, 0, 8, 12, 18, 19 dizisine çevirir (python ile doğrulandı).
  - **Ders sonrası not:** Sunumda BWT çıktısı "aaafhhimst" olarak verildi; bu, sıralı satırların ilk sütunudur, yani karakterlerin sıralanmış hâlidir ve tek başına tersinir değildir. Sıralı döndürmelerin son sütunu "mhfsathiaa"dır ve özgün dize sıralı listede 3. indistedir (0'dan sayarak); python ile doğrulandı. MTF bu doğru çıktıya uygulanırsa 12, 8, 7, 18, 4, 19, 4, 11, 3, 0 elde edilir.

- Şema IV sonuç tablosu (ekrandan, 01:15:05; blok boyutları 1024, 2048, 4096, 8192 byte)
  - Kitab Hibah (5937 byte): 3747 / 3367 / 3213 / 2997. En kötü oran 3747 / 5937 = %63,11.
  - Kitab Thaharoh (118991 byte): 55246 / 48413 / 43196 / 38814. En iyi oran 38814 / 118991 = %32,62. İki oran python ile doğrulandı.
  - Blok boyutu büyüdükçe sıkıştırılmış boyut her dosyada küçülür. Sunan öğrenci makaleyi eleştirirken tablo numaralandırmasında hata olduğunu ve bazı tablo hücrelerinin açıklamasız boş bırakıldığını belirtti.
- IDBE sonuç tablosu (ekrandan, 01:31:55; BPC, BWT / BWT + Star / BWT + IDBE)
  - alice29.txt 2,45 / 2,39 / 2,11; Asyoulik.txt 2,72 / 2,61 / 2,32; cp.html 2,6 / 2,27 / 2,13; fields.c 2,35 / 2,20 / 2,06; grammar.lsp 2,88 / 2,67 / 2,44; kennedy.xls 0,810 / 0,823 / 0,976; lcet10.txt 2,38 / 2,25 / 1,87; plrabn12.txt 2,80 / 2,69 / 2,30; ptt5 0,846 / 0,847 / 0,856; sum 2,80 / 2,75 / 2,89; xargs.1 3,51 / 3,32 / 2,93.
  - Metin dosyalarında IDBE en düşük BPC'yi verir; kennedy.xls, ptt5 ve sum dosyalarında IDBE düz BWT'den kötü sonuç verir (sum'da en iyisi BWT + Star'dır). Süre bakımından IDBE, Star encoding'den genelde hızlıdır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.